# LESSON 14

# 起名 叫它

def cheer():

## 函数定义与调用：给一段代码起个名字

学完你能把常用的几行打包成函数，想用几次就叫几次。

# 同样的几行，要写好几遍?

开机前

**1．检查电源**

**2．检查屏幕**

关机前

**1．检查电源**

**2．检查屏幕**

想再加一步

**每一处都要改**

复制粘贴能做到，可是一要改就得每处都改，漏一处就错。

办法是给这几行起个名字，叫“函数”。要用的时候叫它的名字，改也只改一处。

# def：定义一个函数

hello.py

```
1 def hello():
2     print("你好！ ")
3     print("欢迎来上课")
4 hello()
5 hello()
```

再调用一次，再输出一遍。写一次，用两次。

| 定义 | 不输出 |
| --- | --- |
| 第 4 行 | 你好！<br>欢迎来上课 |
| 第 5 行 | 你好！<br>欢迎来上课 |

# 猜一猜：这段程序输出什么？

guess.py

```
1 def hello():
2     print("你好！")
3 print("结束")
```

结束

只输出“结束”。定义函数就像把菜谱写好放进抽屉，没人照着做，就不会有菜。
函数里的话，只有调用 hello() 的时候才会说。

# # 写函数分三部分

## 1

### 起名字

`def hello():`

def、名字、括号、冒号，一个都不能少

## 2

### 写函数体

下面缩进 4 个空格

缩进的几行都归这个函数

## 3

### 调用

`hello()`

不缩进，名字加括号

名字的起法和变量一样：英文字母开头，看名字就知道它做什么。

# 小测：一共输出几行？

几行？

```
1 def f():
2     print("a")
3     print("b")
4 f()
5 f()
6 f()
```

1. 2 行
2. 3 行
3. 6 行
4. 8 行

答对了！调用 3 次，每次输出 a 和 b 两行，一共 6 行。

# 例题：跳进去，再回来

check.py

```
1 def check():
2     print("1. 检查电源")
3     print("2. 检查屏幕")
4 print("开机前: ")
5 check()
6 print("关机前: ")
7 check()
```

输出“关机前:”，再调用一次，又跳进去做一遍。

开机前:
1. 检查电源
2. 检查屏幕
关机前:
1. 检查电源
2. 检查屏幕

# 动手改一改

只改一处，两次调用都跟着变。

## 检查再加一步

在检查步骤的最后加上“3．检查网线”。开机前和关机前都要检查三步。

输出

```
开机前：
1．检查电源
2．检查屏幕
3．检查网线
关机前：
……（同样三步）
```

教室电脑：打开 `starter/03_check_more.py`

# 猜一猜：先调用，再定义，行吗？

order.py（只阅读）

```
1 hello()
2 def hello():
3     print("你好！")
```

屏幕上

```
NameError: name 'hello' is
not defined
```

不行。第 1 行时还没读到定义，程序不认识 hello，报 NameError。

先定义，后调用。定义一般都写在程序最上面。

# 函数里也能放循环

countdown.py

```
def countdown():
    for i in range(3, 0, -1):
        print(i)
    print("发射! ")
countdown()
print("再来一次")
countdown()
```

再调用一次，整套又来一遍。

```
3
2
1
发射!
再来一次
3
2
1
发射!
```

# 动手改一改

把重复的那一句做成函数，换个名字。

## 用函数唱生日歌

把 `hello` 改成一个叫 `happy` 的函数，它输出“祝你生日快乐”。用它唱出下面四句，不同的那一句直接 `print`。

输出

**祝你生日快乐**
**祝你生日快乐**
**祝你生日快乐，亲爱的朋友**
**祝你生日快乐**

教室电脑：打开 `starter/05_birthday.py`

# 小测：最后一行只写了 hello，会怎样？

会怎样？

```
1 def hello():
2     print("你好！")
3 hello
```

1 输出“你好！”

2 什么也不输出

3 报 NameError

4 输出 hello

答对了！只写名字不加括号，不会调用，也不报错，什么都没发生。

# # 这节课常见的三种错误

划线的写法只阅读，不要照抄

| | | |
|---|---|---|
| ~~hello~~ | **不报错** | 没有括号，没调用 |
| 先调用后定义 | **NameError** | 定义写在最上面 |
| 函数体没缩进 | **IndentationError** | 函数体缩进 4 个空格 |

IndentationError 的意思是“缩进出错”。看到它，先看 def 下面一行有没有缩进。

# 找错：第二句口号怎么跑到最前面了？

bug.py（只阅读）

```
1 def cheer():
2     print("加油！ ")
3 print("我们是最棒的！ ")
4 cheer()
5 cheer()
```

想让每次都喊“加油！我们是最棒的！”，结果输出了“我们是最棒的！加油！加油！”。
点你认为有问题的那一行。

点代码里的一行

# 自己挑战

# 发射练习

读入练习次数 n（1 到 5 的整数）。每次先输出“第 k 次”，再倒数 3、2、1，最后输出“发射！”。
倒数的部分写成一个函数，在循环里调用它。

输入

```
2
```

输出

```
第 1 次
3
2
1
发射!
第 2 次
……
```

先自己写，卡住了再翻下一页。

# 卡住了？一次看一条提示

**提示 1**　倒数函数上面已经写过了，放在程序最上面。

**提示 2**　读入 n，用 for 让 k 从 1 走到 n。每一遍先输出“第 k 次”，再调用函数。

**提示 3**　调用要缩进在循环里，才会每一遍都调用。

# # 造一个你的函数

给一段常常要说的话起个名字，想用就叫它。说什么，你自己定。

### 先做到

定义一个函数，至少调用两次，中间穿插别的话。

### 选你喜欢的

儿歌的副歌、班级口号、游戏开场、广播体操口令。

### 还想更厉害

定义两个函数搭配着用，或者在循环里调用函数。

没有标准答案，能运行、你喜欢就好。

# # 和同桌换着玩

## 1

### 换着运行

先读同桌的程序，猜会输出几行，再运行

## 2

### 说一个喜欢的

告诉同桌，哪个函数名起得最好

## 3

### 提一个想法

哪几行还在重复，能不能也做成函数？

听完再改一版，让程序更短、更好改。

# 说得清吗?

| | | |
|---|---|---|
| Q1 | **定义了函数，为什么没有输出?** | 没有调用，函数体不会运行。 |
| Q2 | **调用完函数，程序接着走哪里?** | 回到调用那一行的下一行。 |
| Q3 | **用函数有什么好处?** | 写一次用多次，要改只改一处。 |

# 速查卡

| | |
|---|---|
| `def hello():` | 定义：起名字，后面有括号和冒号 |
| `    print("你好")` | 函数体：缩进 4 个空格 |
| `hello()` | 调用：名字加括号 |
| `hello` | 没有括号，不会调用 |
| **先定义，后调用** | 定义写在程序最上面 |

# # 这些后面再学

### 参数

下节课：调用时往括号里塞数据，每次做的不一样

### return

下节课：函数算完把结果交回来

### print() len()

它们也是函数，Python 早就写好了

今天的函数每次都做一样的事。下节课让它根据你给的数据，做不一样的事。

# 谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 python-for-youth

下一课：参数与返回值